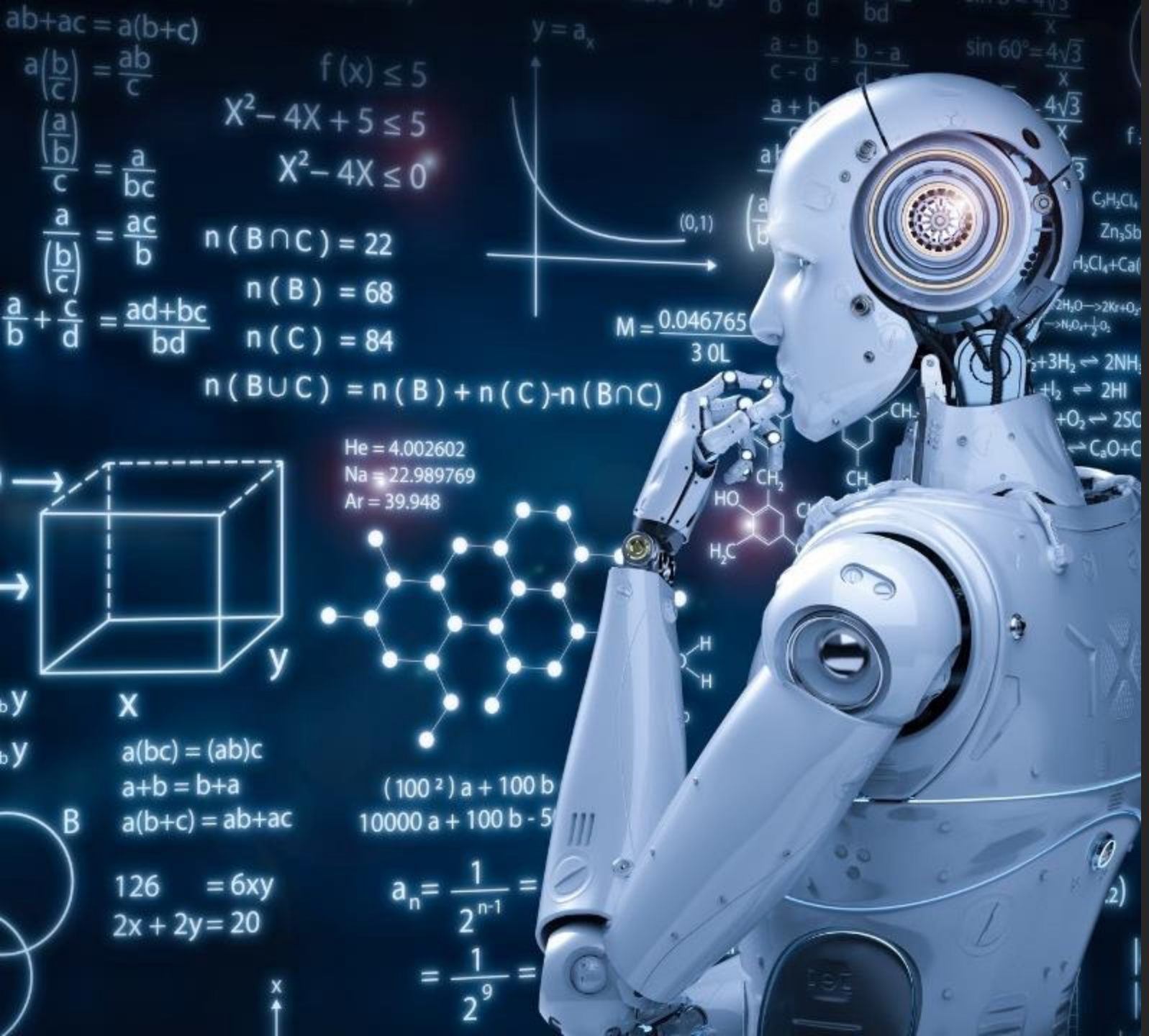


PGR206: Data Structures and Algorithms

Learning Outcome 1

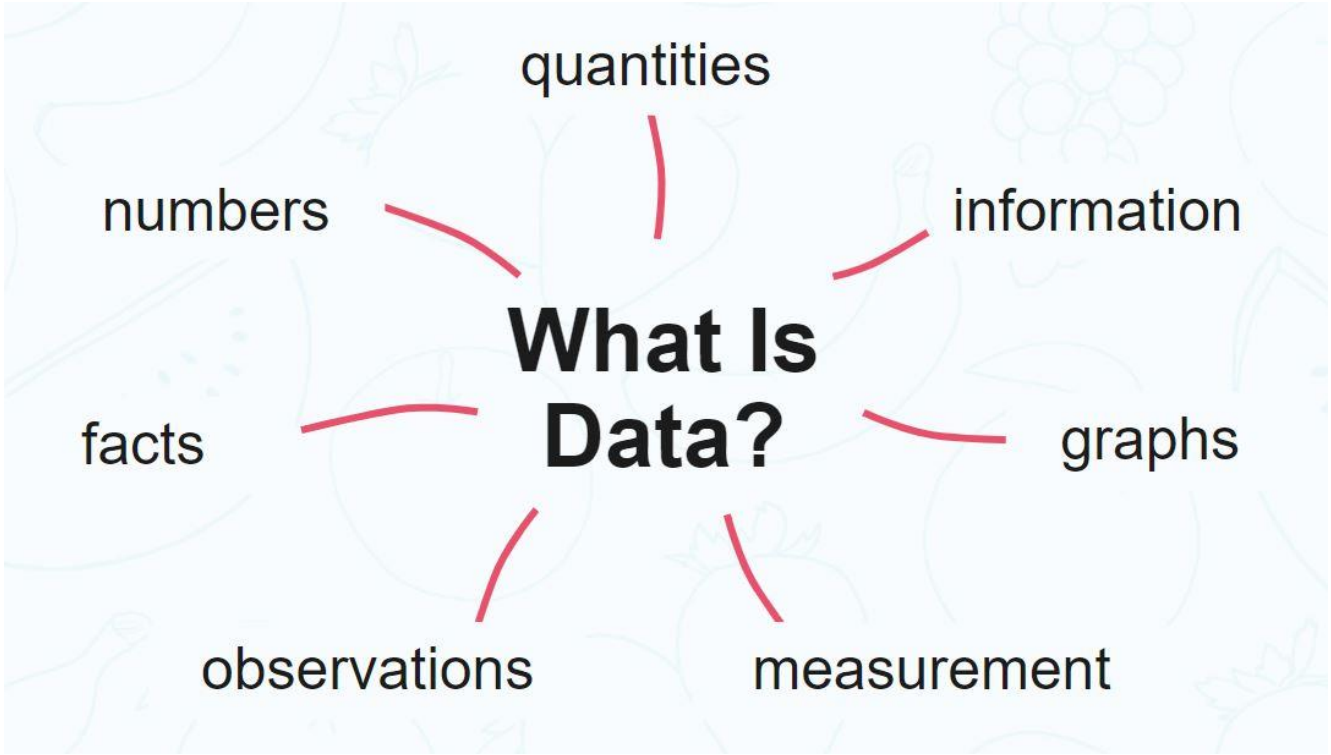
UNDERSTANDING DATA STRUCTURES AND ALGORITHMS

PROF. DR. RASHMI GUPTA

RASHMI.GUPTA@KRISTIANIA.NO

Major Objectives

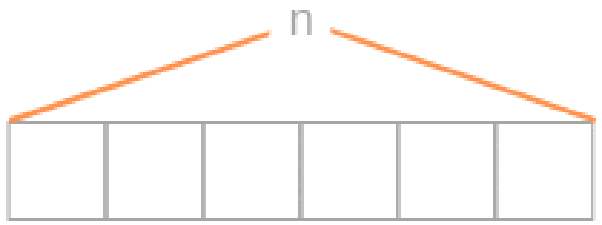
- What is “data”? Why organize it?
- Data Structures? Types? Major operations?
- What is “Problem” and “Algorithm”?
- Analyzing and solving a problem by **approach or methodology: No Programming!!!**
- Algorithm vs Pseudocode vs programming code

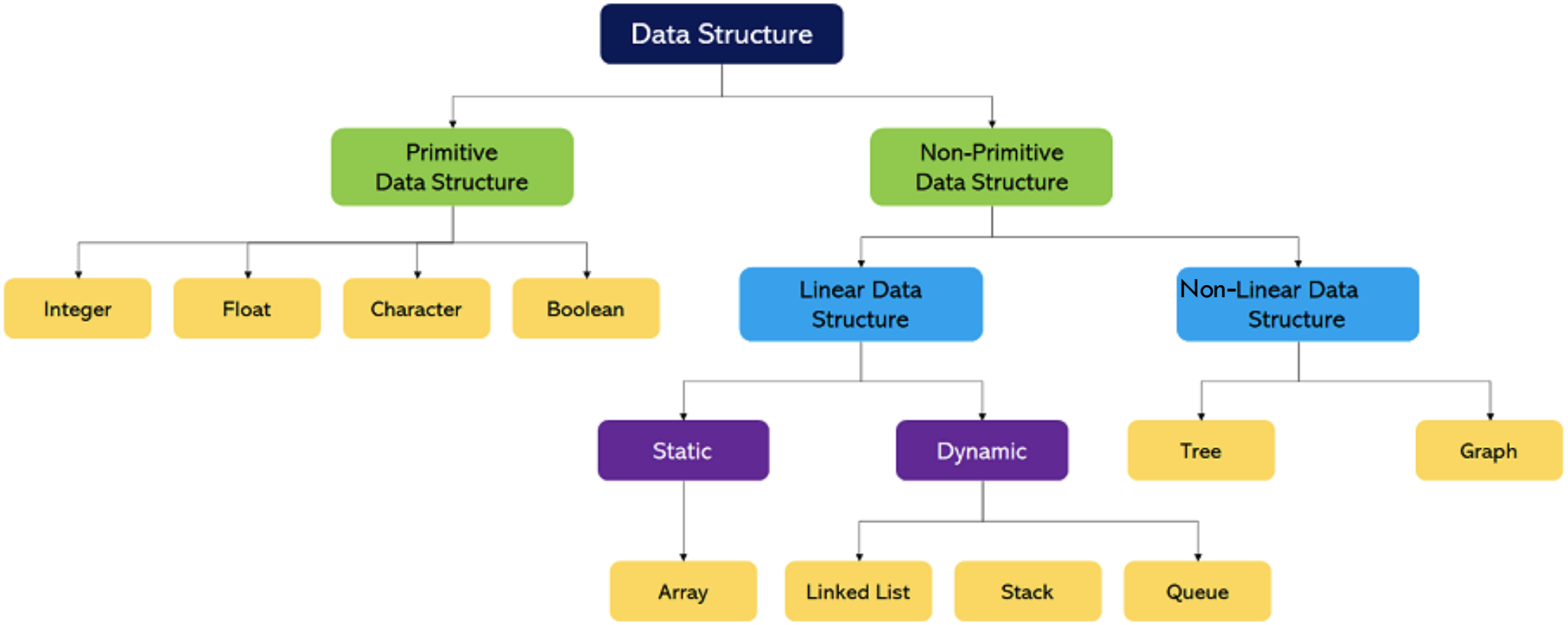


What is “data”? Why organize it? How!!

Data Structure

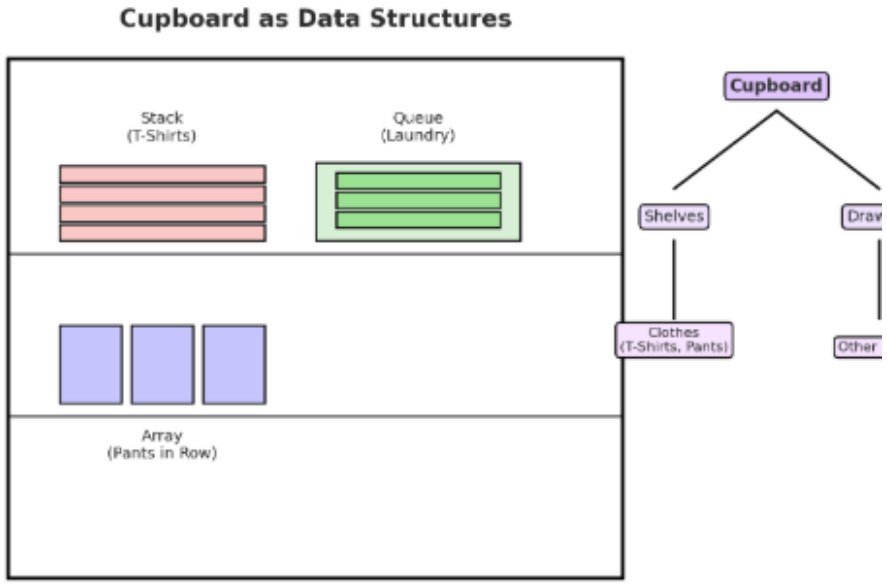
- Data Structure is a way to store and organize data in memory to be used **efficiently**.
- The data structure is not any programming language like C, C++, java, etc. It is a set of algorithms that we can use in any programming language to structure the data in the memory.
- For example, Array
  - is a collection of memory elements in which data is stored sequentially, i.e., one after another.





Types of Data Structures

Example



Perfect example 👕👗 — your cupboard (wardrobe) organization is basically a set of data structures in action:

Stack: If you fold T-shirts and put them in a pile, you usually take the one from the top and add washed ones on top again. This is a LIFO (Last In, First Out) structure.

Queue: If you hang clothes for ironing or laundry in a basket, the first one you put in is usually the first one you take out to iron. That's a FIFO (First In, First Out) queue.

Array (List): If you line up clothes side by side on a shelf (e.g., trousers arranged in order), each has a fixed position, and you can directly grab the 3rd one..

Tree: Cupboard → Section (Hanging, Drawers, Shelves) → Type (Shirts, Pants, Jackets) → Color/Season.

This is a hierarchical structure like a tree.

Major Operations?

Searching: We can search for any element in a data structure.

Sorting: We can sort the elements of a data structure either in an ascending or descending order.

Insertion: We can also insert a new element in a data structure.

Updation: We can also update the element, i.e., we can replace the element with another element.

Deletion: We can also perform the delete operation to remove the element from the data structure.

What is Problem and Algorithm?

Sort a list

S = [10, 7, 11, 5, 13, 8]

Find two different numbers in a list

S = [10, 7, 11, 5, 13, 8] , where x = 5 and y = 20

Now….

Sort a list

S = [10, 7, 11, 5, 13, 8, 38, 37, 14, 92, 84, 74, 77, 20, 40, 47, 33, 65, 62, 69, 73]

Find two different numbers in a list

S = [10, 7, 11, 5, 13, 8, 38, 37, 14, 92, 84, 74, 77, 20, 40, 47, 33, 65, 62, 69, 73], where x = 5 and y = 20

Method to solve a problem

- Independent of
  - programming language
  - Style
- Approach to solving a problem
  - Sequential Search, Binary Search, or any other?
- Concerning Points!
  - Can we solve the problem using the given technique?
  - How efficient is it?
    - Time and Storage!!
- Conclusion: Data Structures allow us to organize and store data, whereas Algorithms allow us to process that data meaningfully and efficiently.

Algorithm (Real-Life Example)

Let's understand the algorithm through a real-world example.

Suppose we want to make a lemon juice, so following are the steps required to make a lemon juice:

- Step 1: First, we will cut the lemon into half.
- Step 2: Squeeze the lemon as much you can and take out its juice in a container.
- Step 3: Add two tablespoon sugar in it.
- Step 4: Stir the container until the sugar gets dissolved.
- Step 5: When sugar gets dissolved, add some water and ice in it.
- Step 6: Store the juice in a fridge for 30 minutes.
- Step 7: Now, it's ready to drink.

The above real-world can be directly compared to the definition of the algorithm. We cannot perform the step 3 before the step 2, we need to follow the specific order to make lemon juice. An algorithm also says that each and every instruction should be followed in a specific order to perform a specific task.

Algorithm (in Programming)

Write an algorithm to add two numbers entered by the user.

- Step 1: Start
- Step 2: Declare three variables a, b, and sum.
- Step 3: Enter the values of a and b.
- Step 4: Add the values of a and b and store the result in the sum variable, i.e., sum=a+b.
- Step 5: Print sum
- Step 6: Stop

Algorithm of linear search :

1. Start from the leftmost element of arr[] and
one by one compare x with each element of arr[].
2. If x matches with an element, return the index.
3. If x doesn't match with any of elements, return -1.

Here, we can see how the steps of a linear search program are explained in a simple, English language.

Pseudocode for Linear Search :

```
FUNCTION linearSearch(list, searchTerm):
     FOR index FROM 0 -> length(list):
       IF list[index] == searchTerm THEN
           RETURN index
       ENDIF
       ENDLOOP
           RETURN -1
END FUNCTION
```

In here, we haven't used any specific programming language but wrote the steps of a linear search in a simpler form which can be further modified into a proper program.

Program for Linear Search :

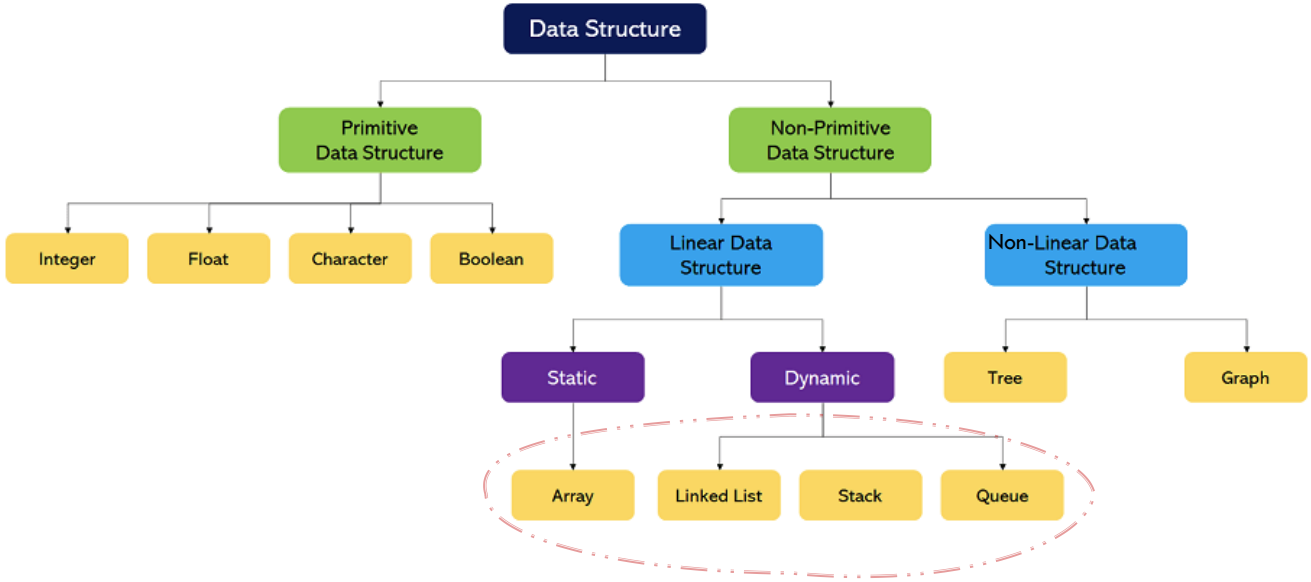
Cpp | Python3

```cpp
// C++ code for linearly search x in arr[].  If x
// is present  then return its  location,  otherwise
// return -1
int search(int arr[], int n, int x)
{
    int i;
    for (i = 0; i < n; i++)
        if (arr[i] == x)
         return i;
    return -1;
}
```

Algorithm, Pseudocode, Program!!!!

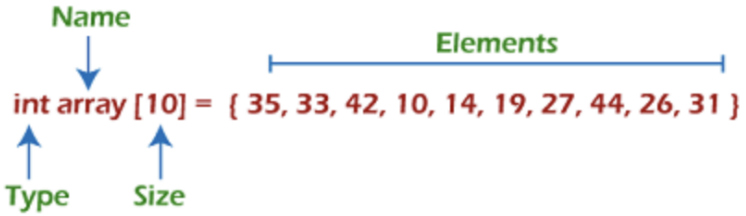
Major Objectives

- Non-primitive linear data structures?
- Array in data structure
- Linked lists in data structure
- Stack in data structure
- Queue in data structure



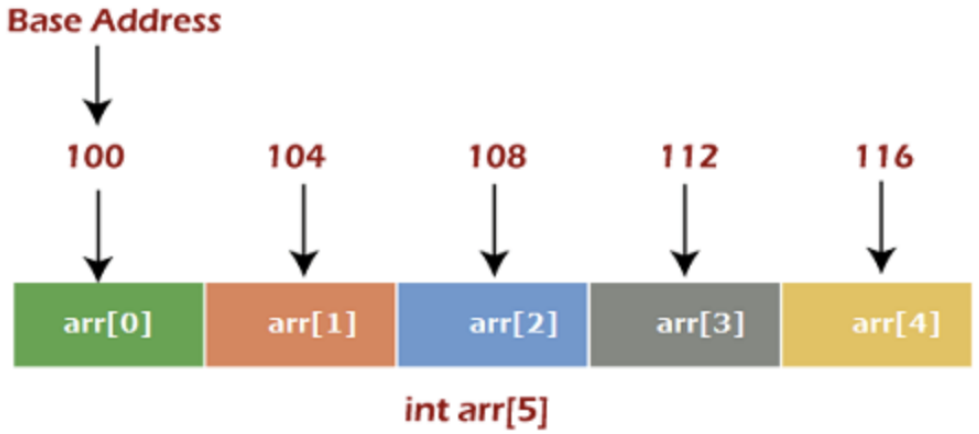
Array: Static Data Structure

- ✓Each element in an array is of the **same data type** and carries the **same size i.e.,** 4 bytes if we use int data type.
- ✓Elements in the array are stored at **contiguous memory locations,** from which the first element is stored at the smallest memory location.
- ✓Elements of the array can be randomly accessed since we can **calculate the address of each element** of the array with the **given base address** and the **size of the data element.**



- ✓ Index starts with 0.
- ✓ The array's length is 10, which means we can store 10 elements.
- ✓ Each element in the array can be accessed via its index.

Calculating Byte Address of Arrays



Memory Allocation of Arrays

How to access an element from the array?

To access any random element from the array -

- ✓ Base Address of the array.
- ✓ Size of an element in bytes.
- ✓ Type of indexing, array follows.
- ✓ **Byte address of element A[i] = base address + size * (i - first index)**

Basic Operations Supported in Array

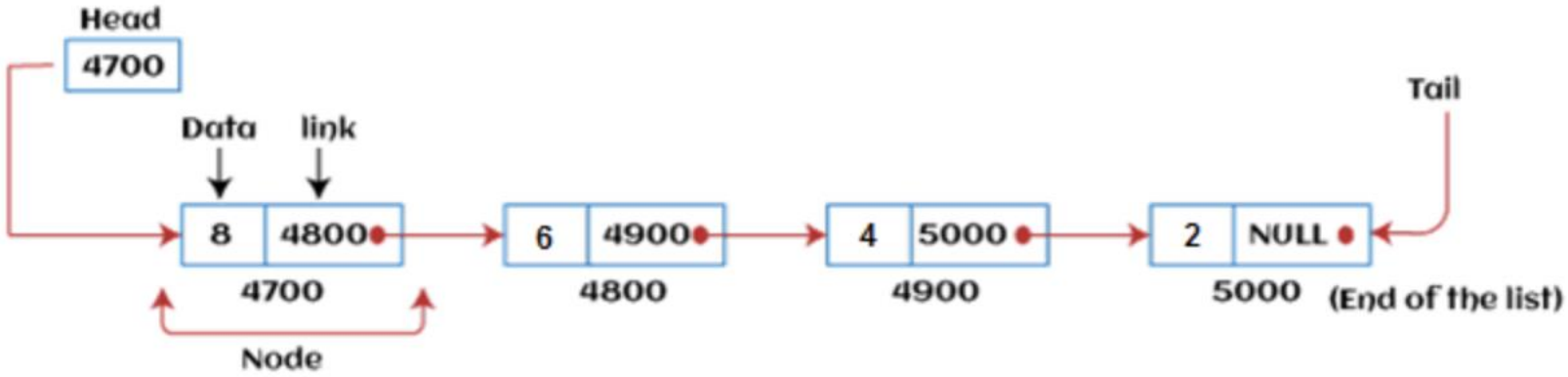
- ✓**Traversal** - This operation is used to print the elements of the array.
- ✓**Insertion** - It is used to add an element at a particular index.
- ✓**Deletion** - It is used to delete an element from a particular index.
- ✓**Search** - It is used to search an element using the given index or by the value.
- ✓**Update** - It updates an element at a particular index.

Lab Tasks : Array Data Structures

- ✓ Reverse Traverse the array (from N to 0).
- ✓ Insert an element in an array:
  - ✓ At the starting array index
  - ✓ At the ending array index
  - ✓ At any random array index
- ✓ Delete an element in an array:
  - ✓ At the starting array index
  - ✓ At the ending array index
  - ✓ At any random array index

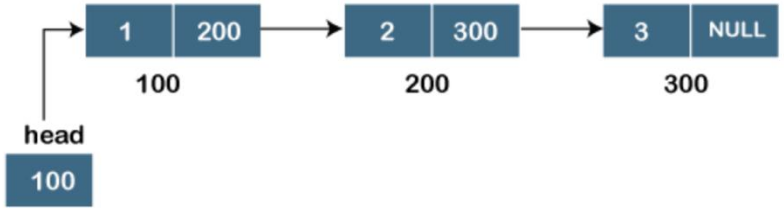
Linked Lists: Dynamic Data Structure

- ✓Linked list is a linear data structure that includes a **series of connected nodes that are randomly stored in the memory.**
- ✓It contains two parts, i.e., the **first** is the **data part**, and the **second** is the **address part.**
- ✓The **last** node of the list contains a pointer to the **null.**

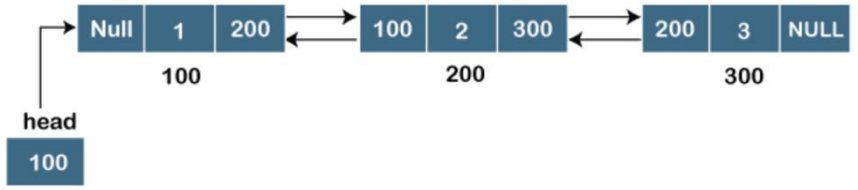


Types of Linked Lists

- ✓Singly-linked lists: Insertion, Deletion, Traversing, Searching Operations
  - ✓In this list, only forward traversal is possible; we cannot traverse in the backward direction as it has only one link in the list.

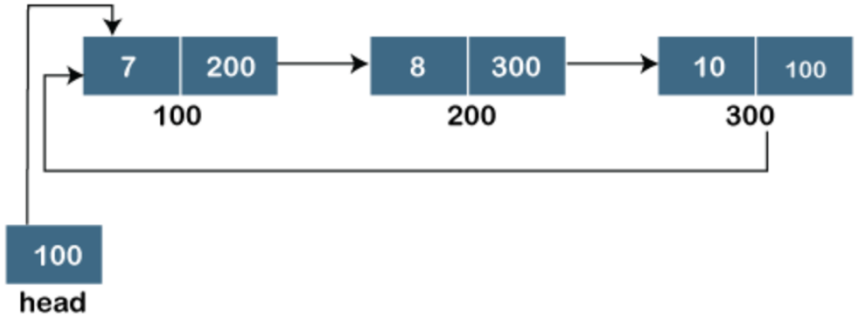


- ✓Doubly linked lists: Insertion, Deletion, Traversing, Searching Operations

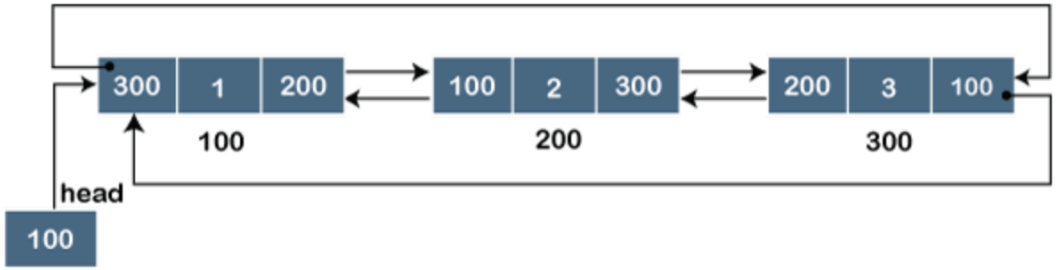


Types of Linked Lists

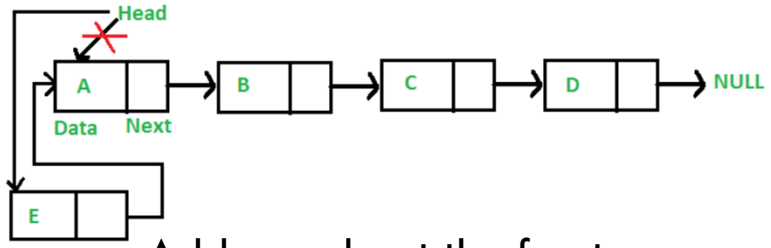
- ✓Circular singly linked lists: Insertion, Deletion, Traversing, Searching Operations



- ✓Circular doubly linked lists: Insertion, Deletion Operations

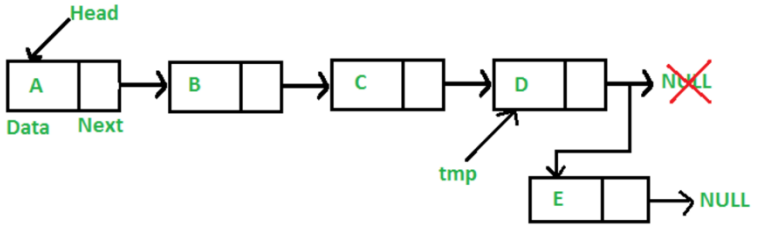


Example of Insertion Operation

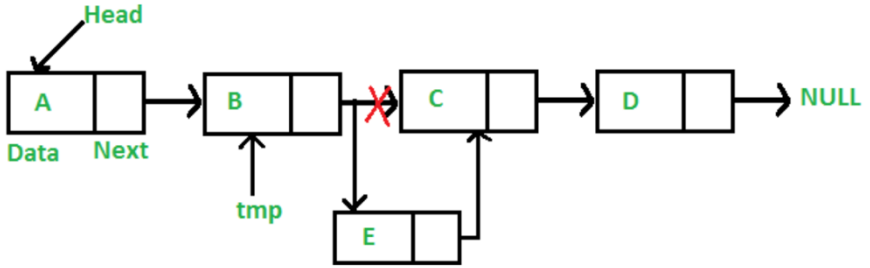


Add a node at the front

1. Allocate the Node
2. Put in the data
3. Make next of new Node as head
4. Move the head to point to new Node



Add a node at the end



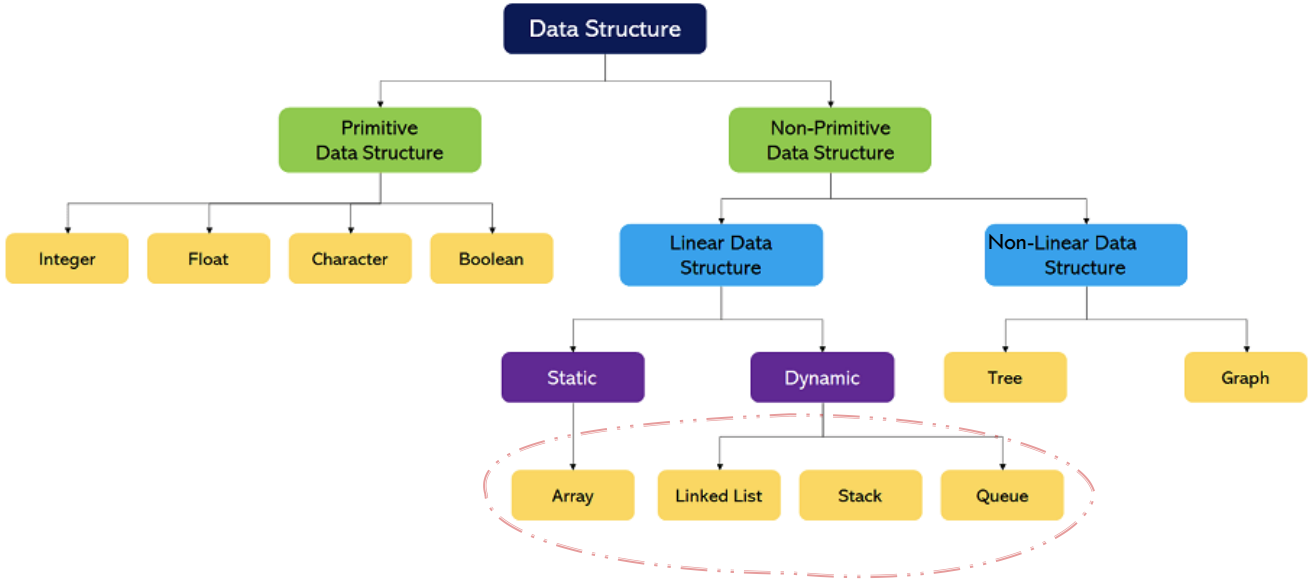
Add a node after a given node

- ✓ Firstly, check if the given previous node is NULL or not.
- ✓ Then, allocate a new node and
- ✓ Assign the data to the new node
- ✓ And then make the next of new node as the next of previous node.
- ✓ Finally, move the next of the previous node as a new node.

Try all other operations of Linked lists in your lab sessions. Take help from StackOverflow or geeksforgeeks websites for any errors and assistance.

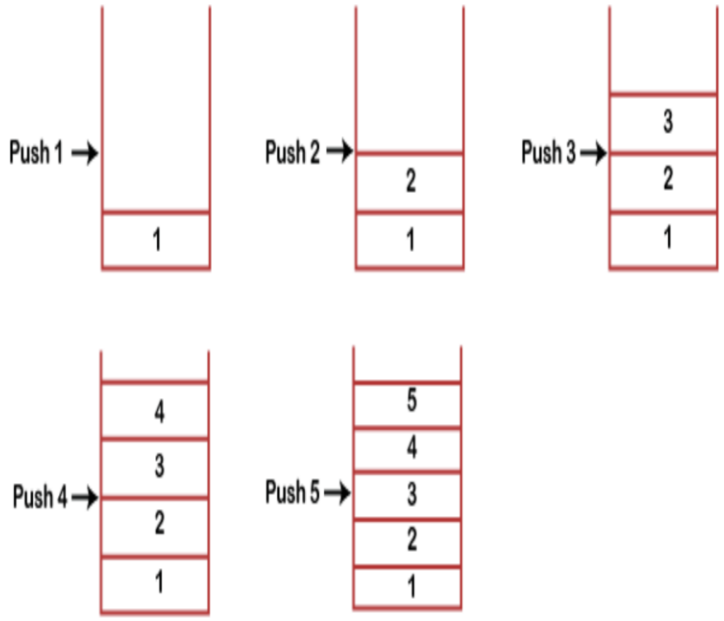
Major Objectives

- ~~Non-primitive linear data structures?~~
- ~~Array in data structure~~
- ~~Linked lists in data structure~~
- Stack in data structure
- Queue in data structure
- Tree in Data Structure
- Graph in Data Structure



Stack: Dynamic Data Structure

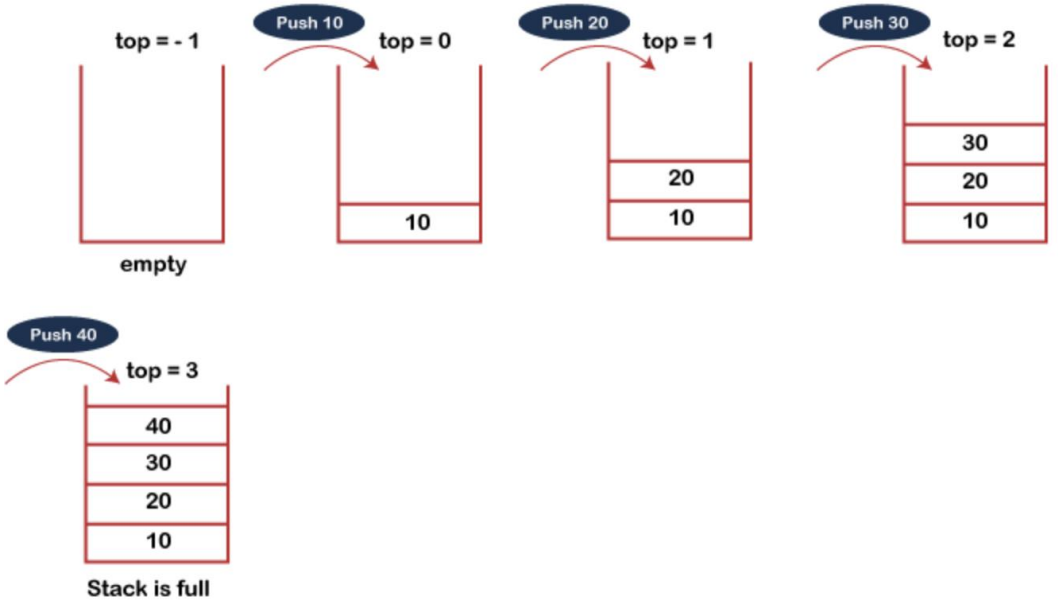
- ✓Stack is a linear data structure that follows the **LIFO (Last-In-First-Out)** principle, which means that the **value entered first will be removed last.** Examples: piles of books, dishes, coffee cups, clothes etc.
- ✓It contains only one pointer top pointer pointing to the topmost element of the stack.
- ✓Whenever an element is added to the stack, it is added on top of the stack, and the element can be deleted only from the top of the stack.
- ✓Stack can be defined as ***a container in which insertion and deletion can be done from one end,*** known as the ***top of the stack.***



Stack Operations

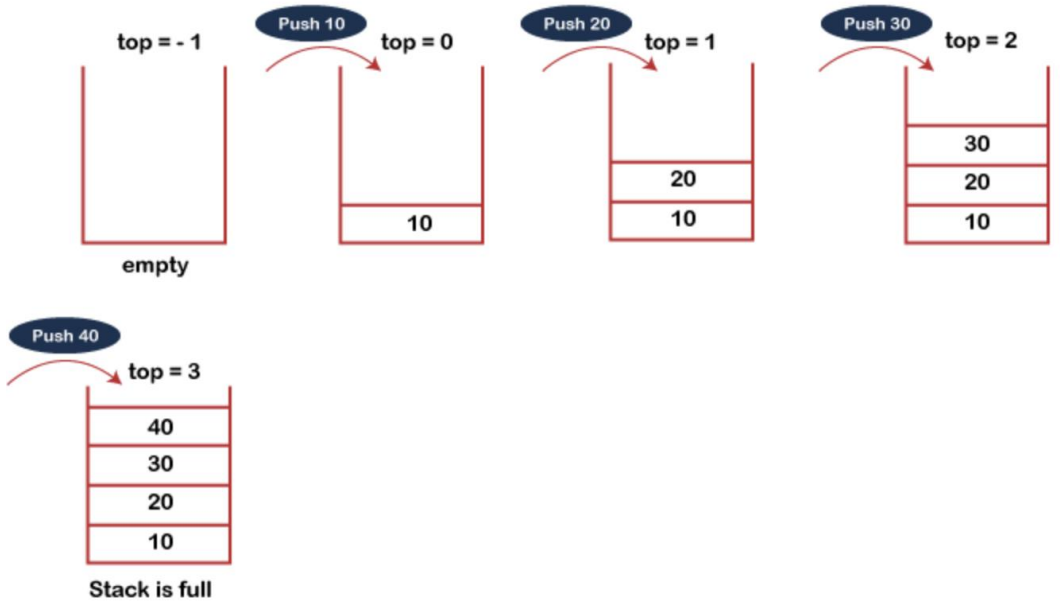
- ✓**push():** When we insert an element in a stack, then the operation is known as a push. If the stack is full then the overflow condition occurs.
- ✓**pop():** When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- ✓**isEmpty():** It determines whether the stack is empty or not.
- ✓**isFull():** It determines whether the stack is full or not.'
- ✓**peek():** It returns the element at the given position.
- ✓**count():** It returns the total number of elements available in a stack.
- ✓**change():** It changes the element at the given position.
- ✓**display():** It prints all the elements available in the stack.

Push Operation



- ✓Before inserting an element in a stack, we **check** whether the **stack is full.**
- ✓If we try to insert the element in a stack, and the stack is full, then the **overflow** condition occurs.
- ✓When we initialise a stack, we **set the value of the top as -1** to **check** that the **stack is empty**.
- ✓When the **new element** is **pushed** in a stack, first, the value of the top gets incremented, i.e., **top=top+1**, and the element will be placed at the new position of the top.
- ✓The elements will be **inserted until we reach the max size** of the stack.

Pop Operation

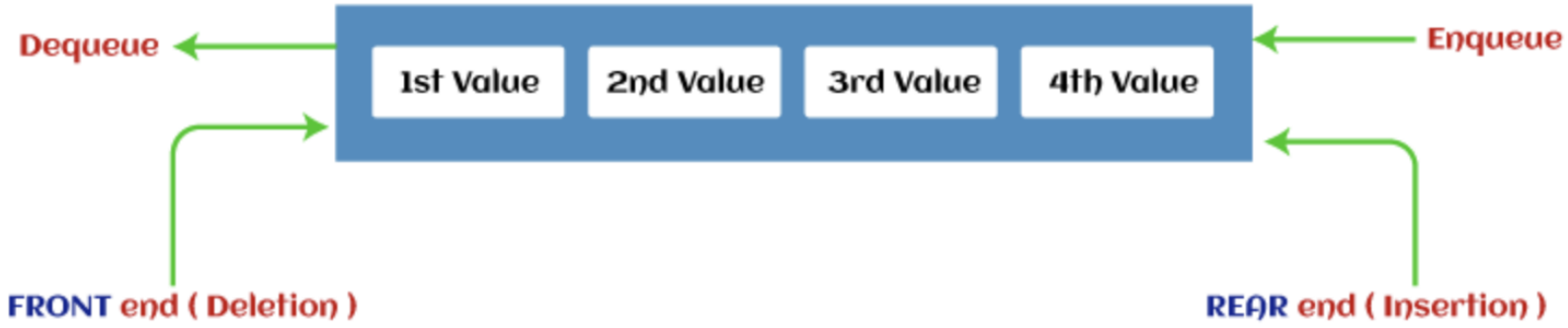


- ✓Before deleting the element from the stack, we **check whether the stack is empty.**
- ✓If we try to **delete** the element from the empty stack, then the ***underflow*** condition occurs.
- ✓If the stack is not empty, we first access the element which is pointed at the ***top.***
- ✓Once the pop operation is performed, the top is decremented by 1, i.e., **top=top-1.**

Lab work: Try array and linked list implementation of the stack and try push (insert) and pop (delete) operations.

Queue: Dynamic Data Structure

- ✓A queue can be defined as an ordered list which enables **insert operations** to be performed at one end called **REAR** and **delete operations** to be performed at another end called **FRONT**.
- ✓Queue is referred to as the **First In, First Out (FIFO)** list principle.
- ✓**For example,** people waiting in line for a rail ticket or movie ticket form a queue.



Types of Queues

✓**Simple queue or linear queue**

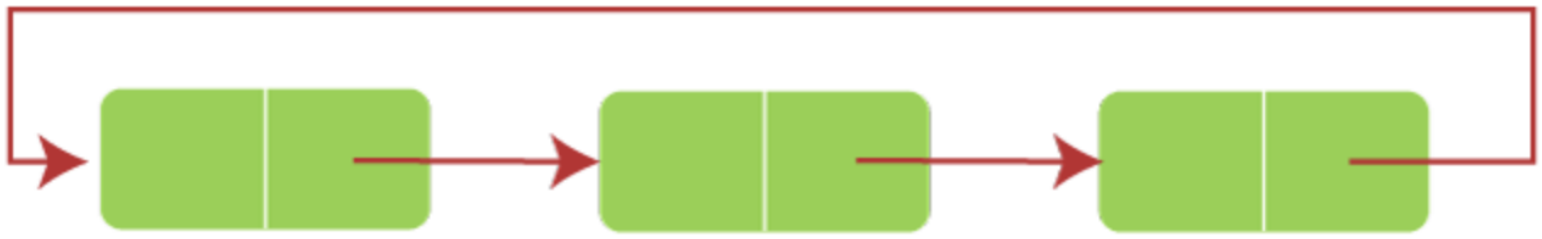
✓An insertion occurs from one end, while the deletion occurs from another. It strictly follows the FIFO rule.

✓Major **drawback** is that **insertion is done only from the rear end**. If the first three elements are deleted from the Queue, we cannot insert more elements even though the space is available in a Linear Queue.

✓In this case, the linear Queue shows the overflow condition as the rear is pointing to the last element of the Queue.

Types of Queues

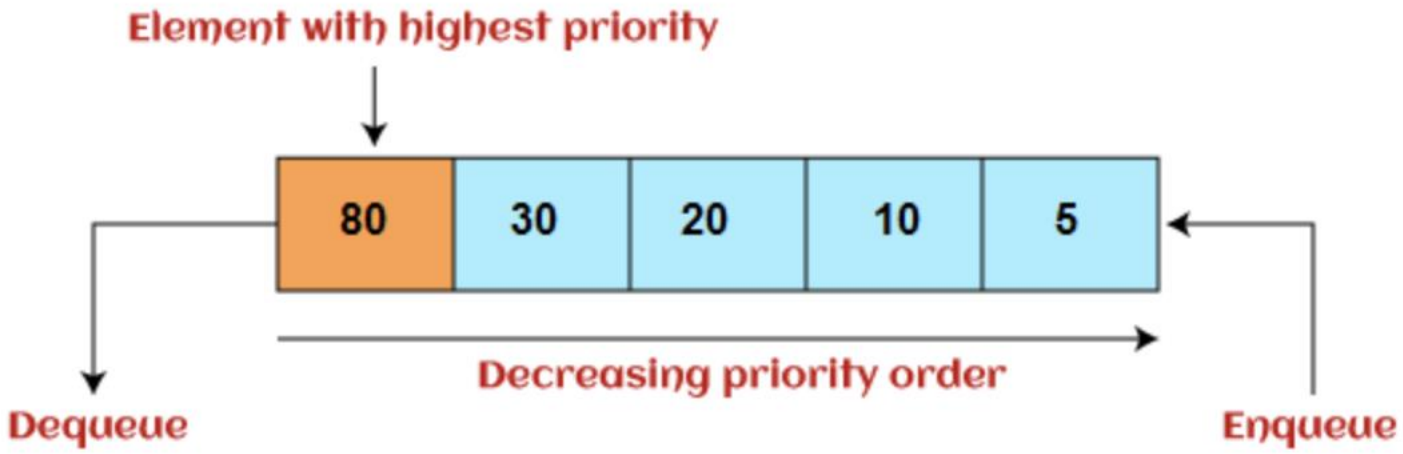
- Circular Queue
  - It is similar to the linear Queue except that the **last element of the queue is connected to the first element.**
  - If the empty space is available in a circular queue, the **new element can be added in an empty space by simply incrementing the value of the rear.**
  - The main **advantage** of using the circular queue is **better memory utilisation. Here is the implementation.**



Circular Queue

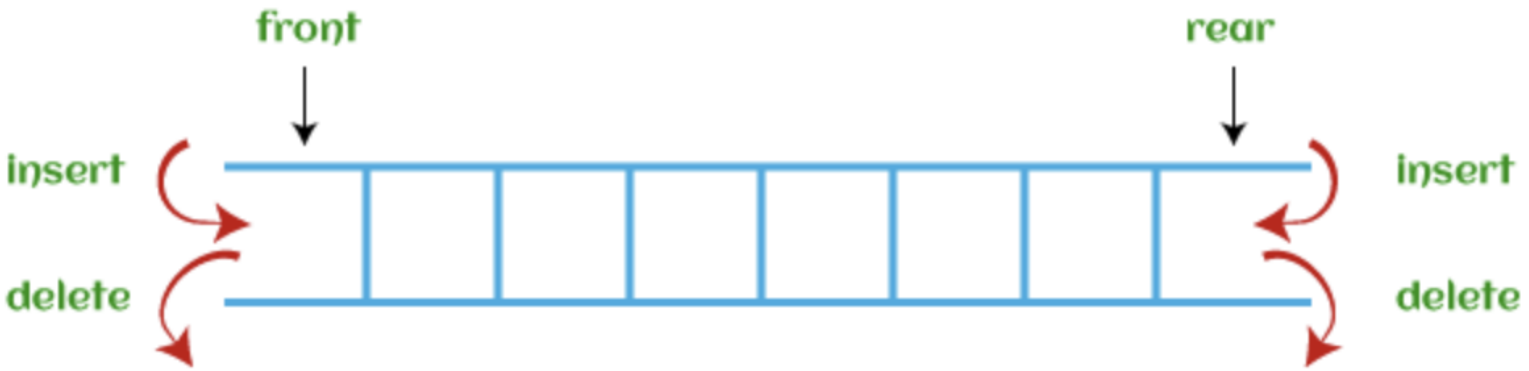
Types of Queues

- Priority Queue
  - It is a **special type of queue** data structure in which **every element has a priority associated with it,** i.e., in **FIFO** principle.
  - **Insertion** in the priority queue takes place **based on the arrival,** while **deletion** in the priority queue occurs **based on the priority.**
  - Priority queue is mainly used to **implement the CPU scheduling algorithms.**



Types of Queues

- ✓Deque (or, Double Ended Queue)
  - ✓In Deque or Double Ended Queue, insertion and deletion can be done from both ends of the queue, either from the front or rear.
  - ✓Deque can be used as a **palindrome checker, meaning that if we read the string from both ends, the string would be the same.**



Queue Operations

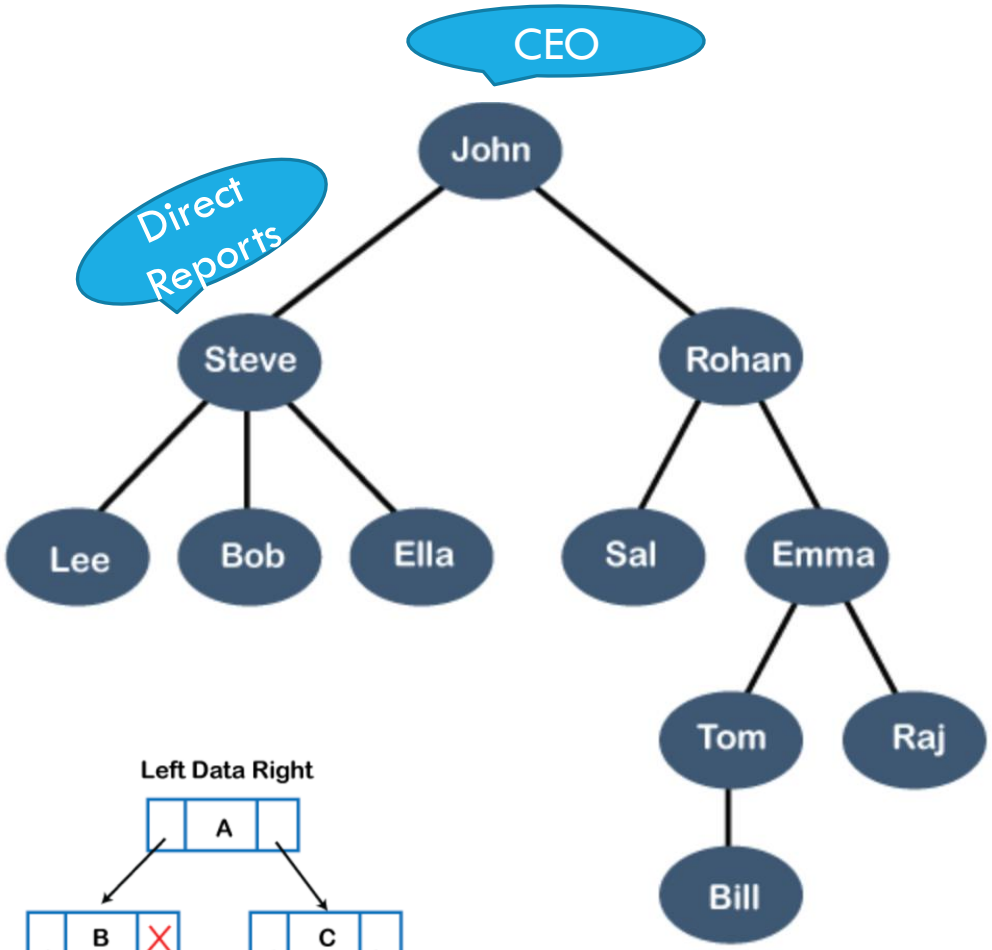
Enqueue: The Enqueue operation is used to insert the element at the rear end of the queue.

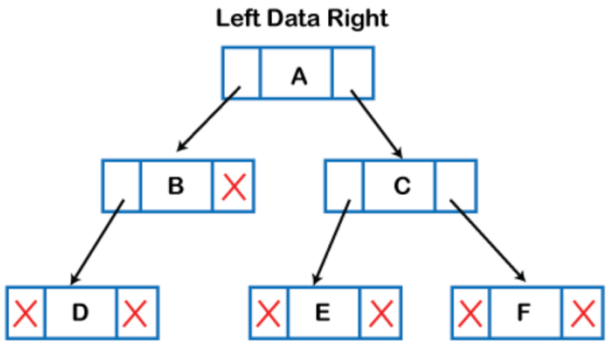
Dequeue: It performs the deletion from the front end of the queue.

Queue overflow (isfull): It shows the overflow condition when the queue is completely full.

Queue underflow (isempty): It shows the underflow condition when the Queue is empty, i.e., no elements are in the Queue.

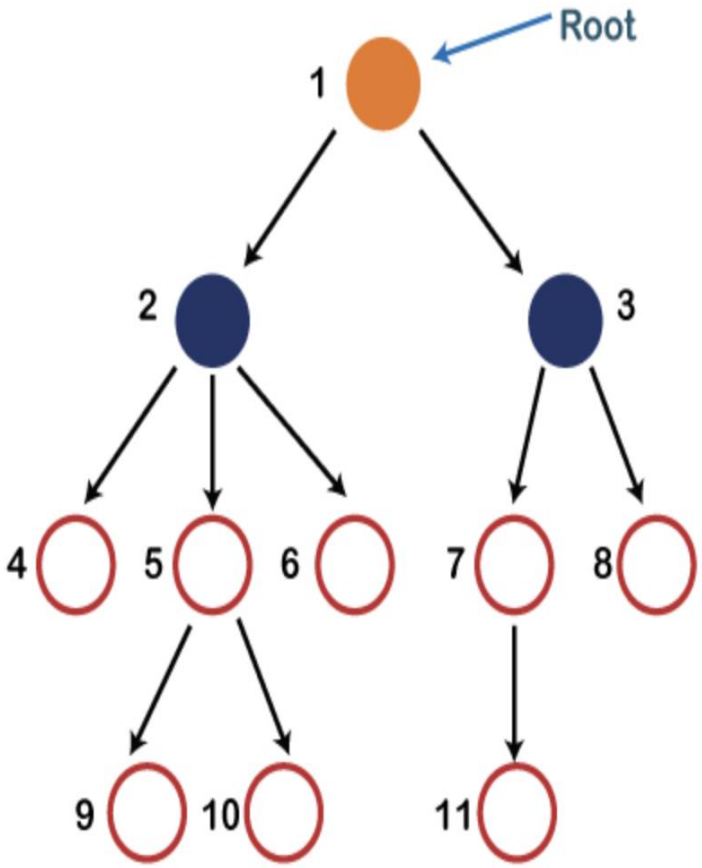
Ways to implement the queues: Array and linked list implementation. In java and python





Tree: Non-Linear Data Structures

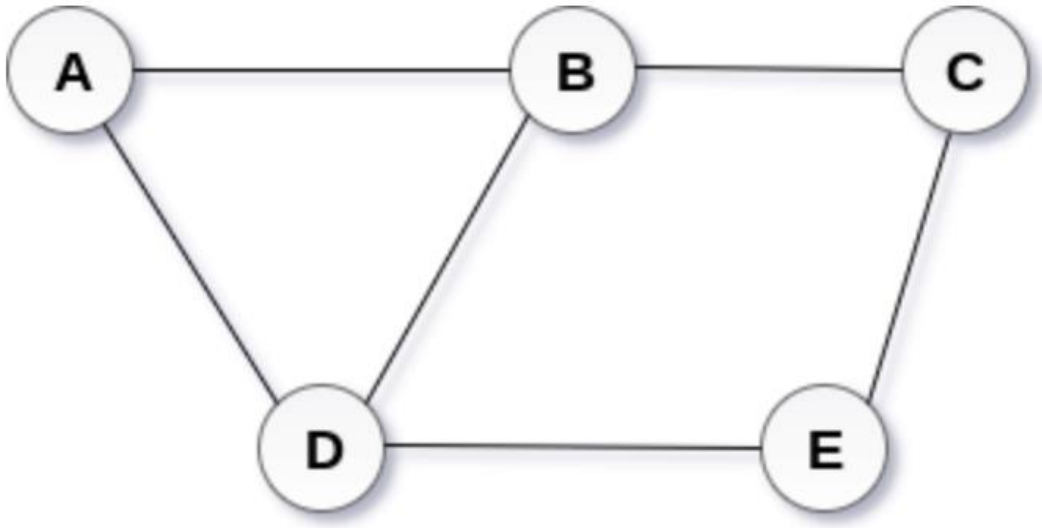
- ✓ This particular **logical structure** is known as a **Tree.** For example, Org hierarchy
- ✓ Its structure is similar to the real tree, so it is named a Tree.
- ✓ In this structure, the **root** is at the **top,** and its branches move downward.
- ✓ Tree data structure is an efficient way of storing data in a hierarchical way.
- ✓ Usually have **Linked list representation** in programming language.



Terms used in Tree

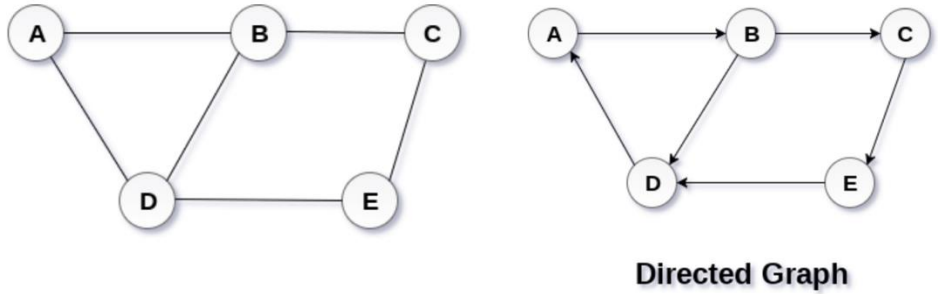
- ✓**Root:** The root node is the topmost node in the tree hierarchy. In other words, the root node is the one that doesn't have any parent. In the above structure, node numbered 1 is the root node of the tree. If a node is directly linked to another node, it would be called a parent-child relationship.
- ✓**Child node:** If the node is a descendant of any node, then the node is known as a child node.
- ✓**Parent:** If the node contains any sub-node, then that node is said to be the parent of that sub-node.
- ✓**Sibling:** The nodes that have the same parent are known as siblings.
- ✓**Leaf Node:-** The node of the tree, which doesn't have any child node, is called a leaf node. A leaf node is the bottom-most node of the tree. There can be any number of leaf nodes present in a general tree. Leaf nodes can also be called external nodes.
- ✓**Internal nodes:** A node has at least one child node, known as an internal
- ✓**Ancestor node:-** An ancestor of a node is any predecessor node on a path from the root to that node. The root node doesn't have any ancestors. In the tree shown in the image, nodes 1, 2, and 5 are the ancestors of node 10.
- ✓**Descendant:** The immediate successor of the given node is known as a descendant of a node. In the figure, 10 is the descendant of node 5.

Graph: Non-Linear Data Structures



- A graph can be defined as group of **vertices and edges** that are used to connect these vertices.
- A graph can be seen as a cyclic tree, where the vertices (Nodes) maintain any complex relationship among them instead of having parent child relationship.
- A graph G can be defined as an ordered set G(V, E) where V(G) represents the set of vertices, and E(G) represents the set of edges which are used to connect these vertices.
- A Graph G(V, E) with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A))

Graph Terminology



- ✓ **Path:** A path can be defined as the sequence of nodes that are followed in order to reach some terminal node V from the initial node U.
- ✓ **Closed Path:** A path will be called as closed path if the initial node is same as terminal node. A path will be closed path if V0=VN.
- ✓ **Connected Graph:** A connected graph is the one in which some path exists between every two vertices (u, v) in V. There are no isolated nodes in connected graph.
- ✓ **Complete Graph:** A complete graph is the one in which every node is connected with all other nodes. A complete graph contain n(n-1)/2 edges where n is the number of nodes in the graph.
- ✓ **Weighted Graph:** In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge e can be given as w(e) which must be a positive (+) value indicating the cost of traversing the edge.
- ✓ **Digraph:** A digraph is a directed graph in which each edge of the graph is associated with some direction and the traversing can be done only in the specified direction.
- ✓ **Adjacent Nodes:** If two nodes u and v are connected via an edge e, then the nodes u and v are called as neighbours or adjacent nodes.